

1e) Yes, the model size can be reduced even more by combining some of the compression methods instead of using each one separately. For my approach, I used the smaller student model from knowledge distillation and then applied full integer int8 quantization to it. This helps shrink the model in two ways: the student model already has fewer parameters than the baseline model, and int8 quantization stores values using 8-bit integers instead of larger 32-bit floating-point numbers.

Compared to the original float32 DNN, the compressed int8 student model should take up much less space and be a better fit for TinyML hardware with limited memory. The main downside is that the accuracy might drop a little because the model is smaller and quantization is less precise. However, if the accuracy is still close to the baseline model, then this is a good tradeoff because the model becomes much smaller while still performing well.

Overall, this shows that using knowledge distillation and quantization together can reduce the model size more than using just one compression method by itself. For a microcontroller deployment, I would choose the smallest model that still keeps acceptable accuracy, since TinyML devices usually have very limited flash, RAM, and processing power.

2.1) Under a free-tier Edge Impulse account, the main learning block types available include supervised learning, unsupervised or anomaly-detection learning, transfer learning, object detection, visual anomaly detection, and classical machine learning. Edge Impulse lists pre-defined learning blocks such as Classification, Regression, Anomaly Detection using K-means, Anomaly Detection using GMM, Visual Anomaly Detection/FOMO-AD, Image Classification with transfer learning, Keyword Spotting with transfer learning, Object Detection using MobileNetV2 SSD FPN, Object Detection using FOMO, and Classical ML.

Examples include:

Learning block type	Example model/use case
Supervised classification	Classifying motion gestures, sounds, images, or sensor states using a Keras classifier
Supervised regression	Predicting a continuous value, such as temperature, pressure, or battery level
Unsupervised/anomaly detection	Detecting unusual machine vibration using K-means or GMM anomaly detection
Transfer learning	Image classification using a pre-trained image model, or keyword spotting using a pre-trained audio model
Object detection	Detecting object locations in images using FOMO or MobileNetV2 SSD FPN
Visual anomaly detection	Detecting visual defects using FOMO-AD with PatchCore or GMM scoring
Classical ML	Classification or regression using models such as XGBoost or LightGBM

2.2) Under the Classifier neural network section, the layer types include Input, Dense, Reshape, Flatten, Dropout, 1D Convolution, 1D Pooling, 2D Convolution, 2D Pooling, and Output layers. Edge Impulse also notes that the simple architecture editor supports predefined layers, while Expert Mode allows access to the TensorFlow/Keras API for more customized architectures.

Layer type	Purpose
Input layer	Receives the input features and passes them into the neural network without performing computation.
Dense layer	A fully connected layer where each neuron connects to all outputs from the previous layer. It is commonly used for classification or regression decisions.
Reshape layer	Changes the shape of the input data without changing the actual data values. This is useful before convolutional layers.
Flatten layer	Converts multi-dimensional data into a one-dimensional vector, usually before Dense layers.
Dropout layer	Randomly disables a fraction of neurons during training to reduce overfitting and improve generalization.
1D Convolution layer	Extracts local patterns from sequential data such as time-series, motion, or audio features.
1D Pooling layer	Reduces the size of 1D feature maps while keeping important information, usually using max or average pooling.
2D Convolution layer	Extracts spatial patterns from 2D data such as images or spectrograms.
2D Pooling layer	Reduces the spatial size of 2D feature maps and makes the model less sensitive to small shifts.
Output layer	Produces the final prediction. For classification, this is usually a softmax output giving probabilities for each class.

2.3) Under the Deployment section, the main compression and optimization options are int8 quantization and the EON Compiler options. Edge Impulse documentation states that quantization changes the model representation from float32 to int8, which can reduce memory use and improve speed, although it may slightly reduce accuracy. The deployment API also lists model type options of float32 and int8, and engine options including tflite, tflite-eon, and tflite-eon-ram-optimized.

The available methods are:

Deployment/compression option	Purpose
Float32 model	Baseline uncompressed model using 32-bit floating-point weights.
Int8 quantization	Compresses the model by using 8-bit integer representation instead of float32. This usually reduces flash/RAM usage and can improve inference speed.
EON Compiler	Optimizes the model for edge deployment by reducing memory and computational requirements.
EON Compiler RAM-optimized mode	A more memory-focused EON deployment option intended to reduce RAM usage on constrained devices.

2.4) For synthetic data generation in the free-tier Studio workflow, the integrated modalities documented by Edge Impulse are image data and audio/keyword-spotting data. Edge Impulse lists a DALL-E Image Generation Block for generating image datasets and a Whisper Keyword Spotting Generation Block for generating keyword-spotting datasets.

Input modality	Synthetic generation method/algorithm
Image	DALL-E image generation block
Audio / keyword spotting	Whisper keyword spotting generation block

Synthetic data is important because it can increase dataset size, add more variation, and cover rare or difficult-to-collect cases. It can also reduce data collection cost and help protect privacy when real data is sensitive. In machine learning, this can improve robustness because the model sees a wider range of examples during training instead of only the limited real-world samples that were collected. Edge Impulse specifically notes that synthetic data can help with cost efficiency, data augmentation, data diversity, and privacy/security.